

# lage práctica 2:

Autores: Héctor Aldao, Guillermo García  
Fecha: 22 de abril de 2026

# Índice

|                                                                     |          |
|---------------------------------------------------------------------|----------|
| <b>1. Configuración del Entorno y Gestión de Sesión</b>             | <b>2</b> |
| 1.1. Detección Dinámica del Master . . . . .                        | 2        |
| 1.2. Optimización de la SparkSession . . . . .                      | 2        |
| <b>2. Ingestión de Datos y Optimización de I/O</b>                  | <b>2</b> |
| 2.1. Uso de Esquema Explícito (StructType) . . . . .                | 2        |
| 2.2. Lectura Distribuida . . . . .                                  | 3        |
| <b>3. Ingeniería de Características mediante Catalyst Optimizer</b> | <b>3</b> |
| 3.1. Transformaciones Cíclicas . . . . .                            | 3        |
| 3.2. Funciones de Ventana para Series Temporales . . . . .          | 3        |
| <b>4. Preparación de Datos y Persistencia en Memoria</b>            | <b>4</b> |
| 4.1. Preprocesamiento para MLlib . . . . .                          | 4        |
| 4.2. Estrategia de Caching . . . . .                                | 4        |
| <b>5. Modelado y Validación Cruzada Paralelizada</b>                | <b>4</b> |
| <b>6. Evaluación y Persistencia de Resultados</b>                   | <b>4</b> |
| <b>7. Resumen de Estrategias de Paralelización</b>                  | <b>5</b> |

## 1. Configuración del Entorno y Gestión de Sesión

El sistema implementa una lógica de inicialización dinámica orientada a la flexibilidad del despliegue en entornos distribuidos.

### 1.1. Detección Dinámica del Master

A través de la función `detect_master_argument`, el script realiza una búsqueda jerárquica de la configuración del clúster:

- Inspección de variables de entorno: `SPARK_MASTER_URL` y `SPARK_MASTER_HOST`.
- Consulta de archivos de configuración locales: `spark-defaults.conf`.
- Estrategia de fallback: En ausencia de parámetros, utiliza `socket.gethostname()` para ejecuciones en nodos maestros de clústeres *standalone*.

### 1.2. Optimización de la SparkSession

La creación de la sesión mediante `create_spark_session` integra configuraciones críticas para el rendimiento:

- **`spark.sql.shuffle.partitions`**: Establecido en 200 para determinar la granularidad de las particiones durante operaciones de re-shuffle.
- **Gestión de Memoria**: Definición explícita de recursos para *driver* y *executor*, mitigando riesgos de errores *OutOfMemory* (OOM) durante la fase de agregación de resultados de validación cruzada.

## 2. Ingestión de Datos y Optimización de I/O

La fase de carga de datos (`load_stock_data`) prioriza la reducción de latencia en operaciones de entrada/salida.

### 2.1. Uso de Esquema Explícito (StructType)

Se evita la inferencia automática de esquemas para eliminar la doble lectura de los archivos fuente, reduciendo el consumo de recursos de I/O a la mitad mediante el uso de `StructType`.

## 2.2. Lectura Distribuida

El motor de Spark asigna de forma balanceada la lectura de múltiples archivos de texto (`data/Stocks/*.txt`) entre los *executors*. Se emplean funciones nativas para el procesamiento de metadatos:

- `F.input_file_name()`: Recupera la fuente de cada registro de forma distribuida.
- `F.regexp_extract`: Aplica expresiones regulares para la extracción de identificadores (*symbols*) directamente en los nodos de ejecución.

## 3. Ingeniería de Características mediante Catalyst Optimizer

### 3.1. Transformaciones Cíclicas

Para capturar la periodicidad temporal, se aplican transformaciones trigonométricas (seno y coseno) sobre las variables de calendario. Estas operaciones se ejecutan como expresiones de columna de bajo nivel optimizadas por el motor Catalyst de Spark.

### 3.2. Funciones de Ventana para Series Temporales

Se emplean `Window Functions` para el cálculo de variables dependientes e independientes:

- **Particionamiento**: Uso de `Window.partitionBy("Symbol").orderBy("Date")` para garantizar la co-ubicación de datos del mismo activo en la misma partición física.
- **Generación de Target**: Implementación de `F.lead(Çlose", 20)` para la definición del objetivo de regresión.
- **Retardos Temporales**: Uso de `F.lag(Çlose", n)` para la extracción de retornos históricos.

## 4. Preparación de Datos y Persistencia en Memoria

### 4.1. Preprocesamiento para MLlib

El pipeline realiza una limpieza de registros incompletos derivados de las ventanas temporales y renombra la variable objetivo a `label`, cumpliendo con los requisitos de la API de Spark MLlib.

### 4.2. Estrategia de Caching

Tras la división estocástica distribuida (`randomSplit`), se invoca el método `train_df.cache()`. Esta operación es fundamental, ya que almacena los datos transformados en la memoria de los *executors*, evitando la recomputación de las funciones de ventana y la re-lectura de disco durante las múltiples iteraciones del *Grid Search*.

## 5. Modelado y Validación Cruzada Paralelizada

El pipeline de aprendizaje automático (`pyspark.ml.Pipeline`) integra las siguientes etapas técnicas:

- **VectorAssembler**: Consolidación de características escalares en vectores densos.
- **StandardScaler**: Normalización de datos dentro del flujo del pipeline para evitar el *data leakage* durante la validación cruzada.
- **CrossValidator y ParamGridBuilder**: Automatización de la optimización de hiperparámetros.
- **Paralelismo de Entrenamiento**: Se establece el parámetro `parallelism=4`, permitiendo que Spark entrene de forma concurrente múltiples configuraciones de modelos, aprovechando los vCores disponibles en el clúster.

## 6. Evaluación y Persistencia de Resultados

Se utiliza el `RegressionEvaluator` para el cálculo distribuido de métricas de error (*RMSE*, *MAE* y  $R^2$ ). La persistencia final se realiza mediante la

recolección selectiva de métricas (`avgMetrics`) hacia el *driver*, permitiendo la interoperabilidad con Pandas para la generación de informes locales sin comprometer la estabilidad de la memoria del nodo maestro.

## 7. Resumen de Estrategias de Paralelización

- **Nivel de Datos:** Ingestión mediante Data Source V2 y transformaciones perezosas (*lazy evaluation*).
- **Nivel de Estructura:** Re-particionamiento por clave lógica para optimizar *Window Functions*.
- **Nivel de Modelo:** Ejecución concurrente de tareas de entrenamiento mediante la orquestación de `CrossValidator`.
- **Nivel de Memoria:** Persistencia distribuida para algoritmos iterativos.